1.Application overview

Foodies is a food delivery application which is dynamic not static which is not limited to users(ordering food)also used by restaurant to accept order and also by delivery members.

Evidence 1 (Persistent Changing State): Placing an order creates a persistent record that updates its status (*Created* → *Accepted* → *Delivered*) across device changes and refreshes.

Evidence 2 (Personalized Interactive Input): User actions (location search, building a cart, payment) prompt the server to validate, compute, and return dynamic, customized feedback.

2.Actors and requirement

- **5 Actors:** Customer, Restaurant Partner, Delivery Partner, Support Agent, and Payment Gateway.
- **Functional Requirements (FRs):**
    1. The system must let a customer browse available restaurants for their location.
    2. The system must let a customer place an order using items from an available menu.
    3. The system must let a customer pay for an order using an external payment method.
    4. The system must let a restaurant partner accept or reject an incoming order.
    5. The system must let a delivery partner view an assigned order for pickup.
    6. The system must let a customer track the real-time status of their placed order.

3. Component Responsibilities & Implementation Claims

Frontend: Presents UI, collects user inputs, displays live feedback (must not be trusted for price or authorization decisions).

Backend: Enforces business logic, authenticates requests, revalidates cart pricing, coordinates order lifecycles.

Database: Persists core data (users, menus, orders, transaction states).

Payment Gateway (External): Authorizes or declines financial transactions securely outside the core app boundary.

Inference Labeling: explicitly label internal architecture choices as *justified inferences* (e.g., *"Inference: The backend uses persistent database storage to maintain state across client restarts"*).

4. Trace the "Place Order" Journey

Step 1 (User Action & Frontend Work): Customer selects items, builds a cart, and clicks "Place Order". Frontend collects the cart data.

Step 2 (Backend Decision): Frontend sends an HTTP request. Backend authenticates the user, checks menu availability, and verifies pricing.

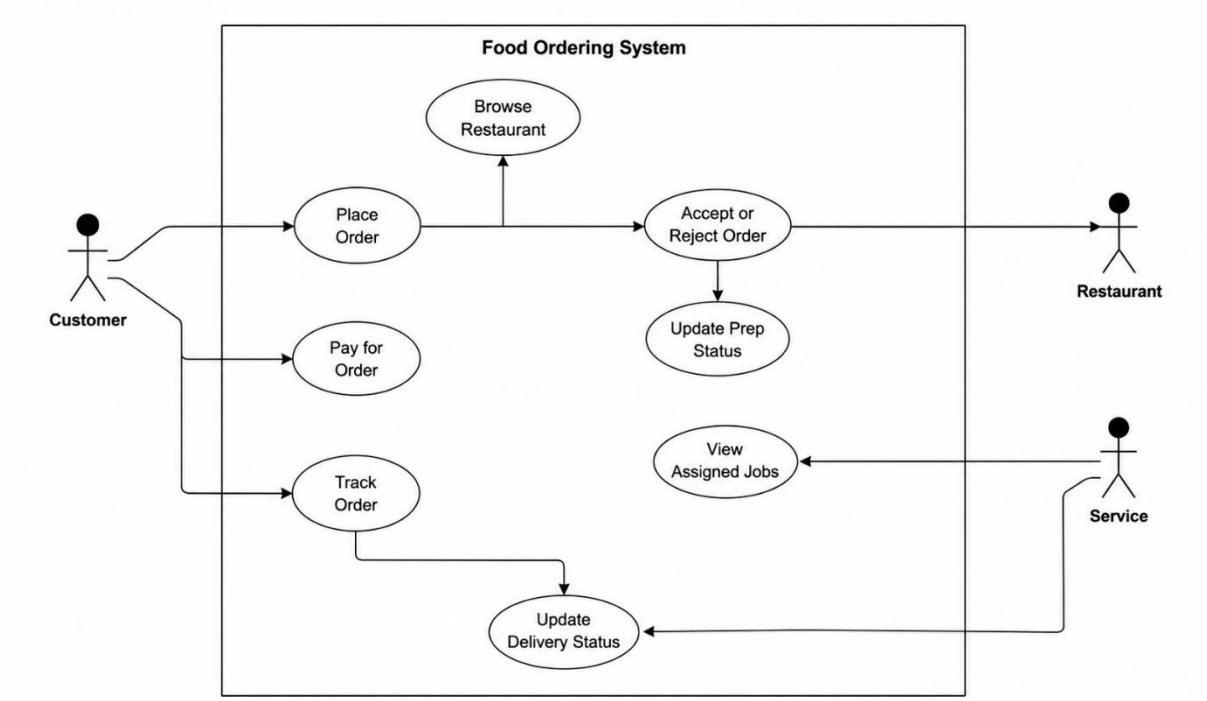
Step 3 (Persistent State Change): Upon payment authorization, the backend creates an order record in the database with status Created.

Step 4 (Visible Outcome): Backend sends a success response; Frontend renders an order confirmation screen with tracking options.

5. Problem Analysis (Testing Edge Cases)

- **Scenario A: Menu price changes while item is in cart**
    - *Who Detects:* Backend during checkout validation.
    - *Who Decides:* Backend (rejects stale price).
    - *State Changes:* No order state created; cart remains uncommitted.
    - *User Sees:* UI error alert showing updated prices and prompting review.
- **Scenario B: Payment succeeds, but server times out before order confirmation**
    - *Who Detects:* Backend/Payment reconciliation service.
    - *Who Decides:* Backend (holds transaction in a Pending Reconciliation state).
    - *State Changes:* Database records transaction outcome without duplicating the order.
    - *User Sees:* UI message stating "Verifying payment status..." instead of a failed error.

6. Use Case Diagram



7. Consistency Review

- **Identify a Contradiction/Gap:** Mention a discrepancy caught during drafting (e.g., "Initial draft had the frontend verifying item availability, contradicting the backend's role as the sole authority on business rules").

- **Explain the Correction:** Explain how it was resolved (e.g., "Updated the journey and component sections so the frontend only collects cart data while the backend revalidates item stock against the database before order creation").